CS5494 Topics in Generative AI - Assignment 1 Report

Group members: `[Name (Student ID)]`, `[Name (Student ID)]`, `[Name (Student ID)]`

Code / data link: `[Insert shared Colab / GitHub / cloud link here]`

---

1. Introduction

This assignment implements and evaluates three generative models for handwritten digit images:

1. **Fully Visible Sigmoid Belief Network (FVSBN)** - an autoregressive model where every pixel is a logistic regression conditioned on all previous pixels.
2. **Neural Autoregressive Distribution Estimator (NADE)** - a feed-forward neural autoregressive model that shares parameters across conditionals for efficiency.
3. **Conditional Generative Adversarial Network (cGAN)** - an adversarial generator trained on the original (non-binarised) MNIST images, conditioned on digit labels.

The first two models are trained on binarised MNIST and optimise the exact negative log-likelihood, while the GAN is trained on the original MNIST pixel values with a label-conditioned generator/discriminator. We report the model design, training procedure, generated samples, loss curves, training-progress visualisations and runtimes.

2. Methods

2.1 Data pre-processing

- **FVSBN / NADE:** MNIST images are converted to `[0, 1]` tensors and binarised by treating each pixel value as a Bernoulli probability (`torch.bernoulli`). Binarisation is applied on-the-fly for every minibatch, which acts as a form of dynamic data augmentation.
- **GAN:** The original MNIST images are only rescaled to `[0, 1]` with `transforms.ToTensor()`; no binarisation is applied, as required by Task 1.3.
- The full training set (60,000 images) is used for all models, with 10,000 test images reserved for likelihood-based evaluation.

2.2 FVSBN

The FVSBN factorises the joint distribution as

$$p(\mathbf{x}) = \prod_{i=1}^{784} p(x_i \mid x_{<i}),$$

where each conditional is a logistic regression

$$p(x_i = 1 \mid x_{<i}) = \sigma\left(\sum_{j<i} W_{ij} x_j + b_i\right).$$

The autoregressive constraint is enforced with a strictly lower-triangular mask, so pixel *i* only depends on previously ordered pixels. The model is trained by minimising the per-pixel negative log-likelihood (equivalently BCE with logits), and results are reported in bits per dimension (bpd). Sampling is sequential: each pixel is drawn from its Bernoulli conditional given the already sampled pixels.

2.3 NADE

NADE also factorises the joint as an autoregressive model, but replaces the independent logistic regressions with a shared feed-forward network:

$$h_i = \sigma\left(W_{<i} x_{<i} + c\right), \qquad p(x_i = 1 \mid x_{<i}) = \sigma\left(V_i h_i + b_i\right).$$

The key efficiency trick is that all hidden states h_i for a training example can be computed in one pass using cumulative pre-activations of the shared weight matrix W. This keeps the model a one-hidden-layer MLP in the forward pass while still defining a valid autoregressive density.

2.4 Conditional GAN

The generator maps a latent vector z and a class label y to a 28x28 image $G(z, y)$, while the discriminator maps an image and a label to a real/fake logit $D(x, y)$. Labels are embedded as 10-dimensional vectors and concatenated with the input of each network. The minimax objective is

$$\min_G \max_D \; \mathbb{E}_{(x,y)}\log D(x,y) + \mathbb{E}_{z,y}\log(1 - D(G(z,y),y)).$$

Two stabilisation techniques are used. First, real labels are smoothed to 0.9 and small Gaussian noise is added to discriminator inputs, preventing the discriminator from saturating. Second, real images paired with a *wrong* label are also treated as fake; this forces the discriminator to use the label and helps avoid mode collapse. Without conditioning and the wrong-label penalty, the vanilla GAN collapsed to generating a single digit class.

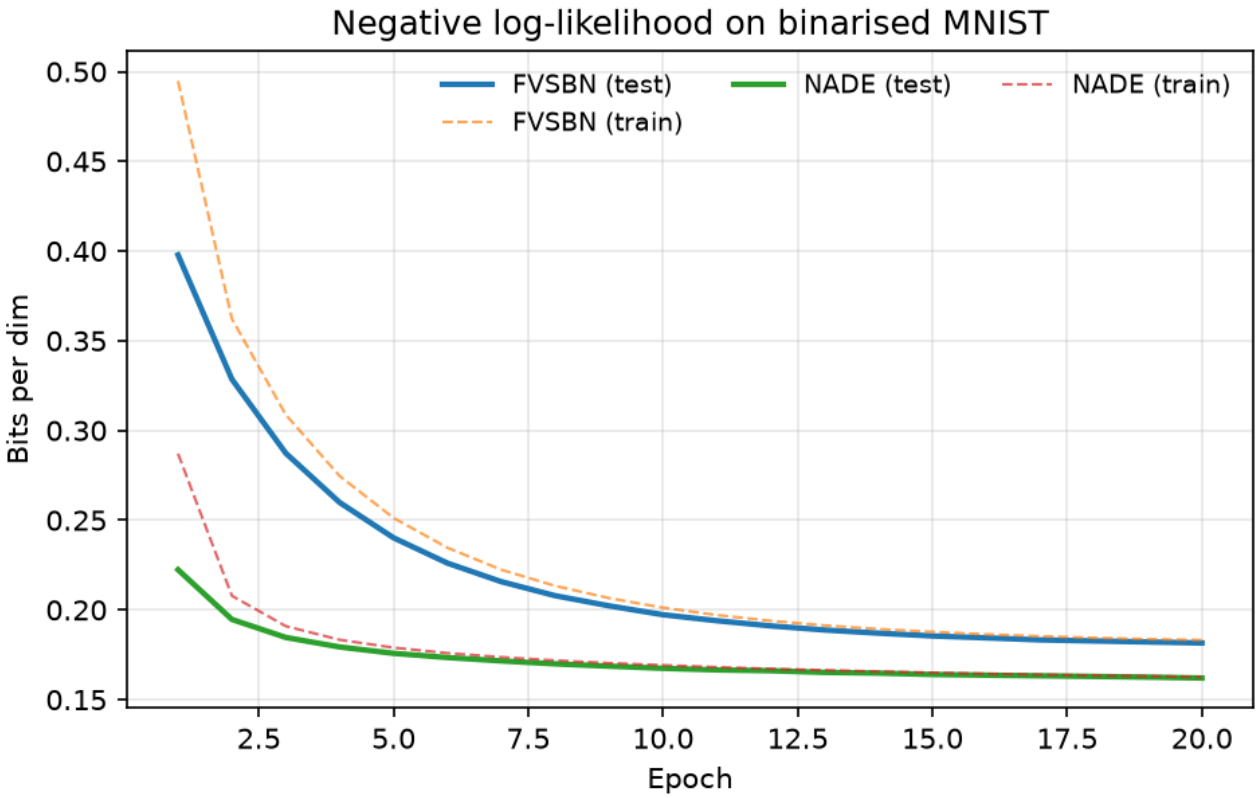
3. Experiments and results

3.1 Hyperparameters

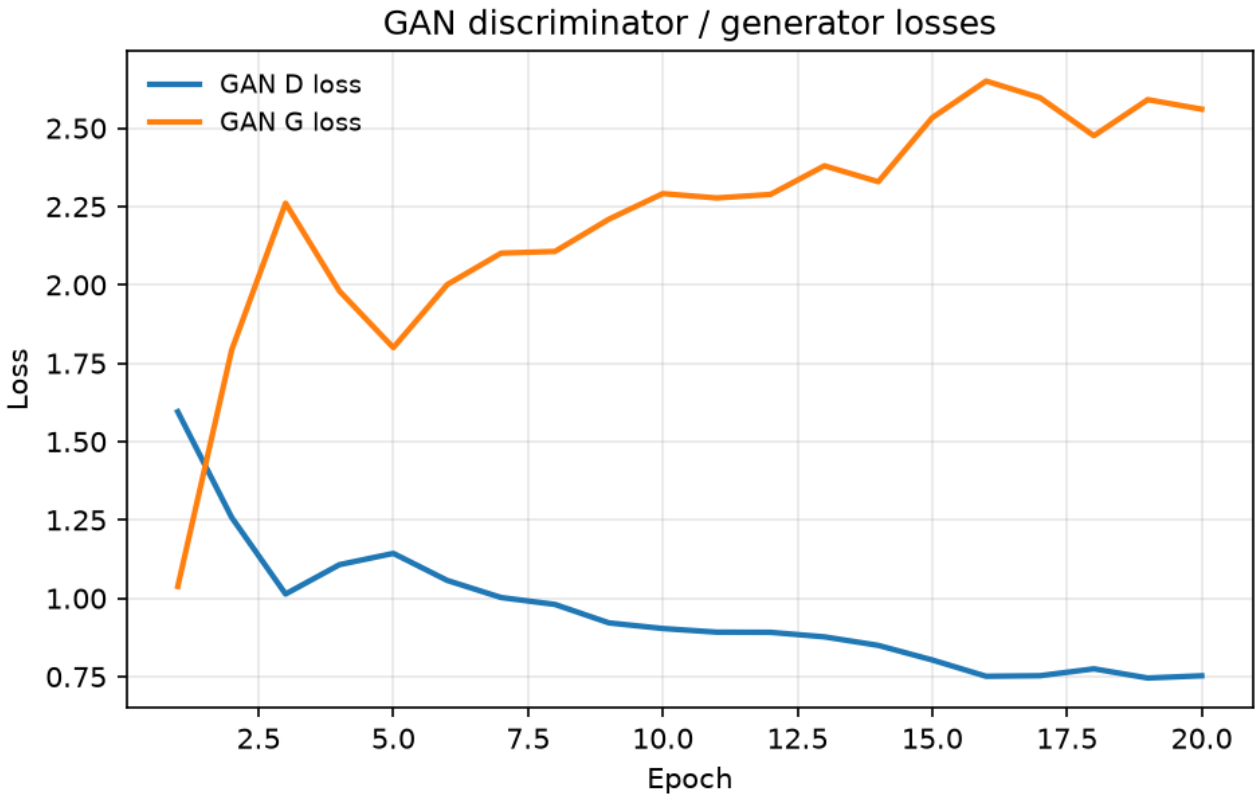
Setting	FVSBN	NADE	cGAN
Data	Binarised MNIST	Binarised MNIST	Original MNIST
Training set	60,000	60,000	60,000
Batch size	128	128	128
Epochs	20	20	20
Optimiser	Adam	Adam	Adam (beta=0.5, 0.999)
Learning rate	1e-3	1e-3	G 1e-4 / D 1e-4
Hidden size / latent dim	-	500	z-dim 100
Trainable parameters	615,440	785,284	G 1,489,012 / D 236,645

3.2 Loss curves

The likelihood models are compared with bits-per-dimension on both train and test splits.

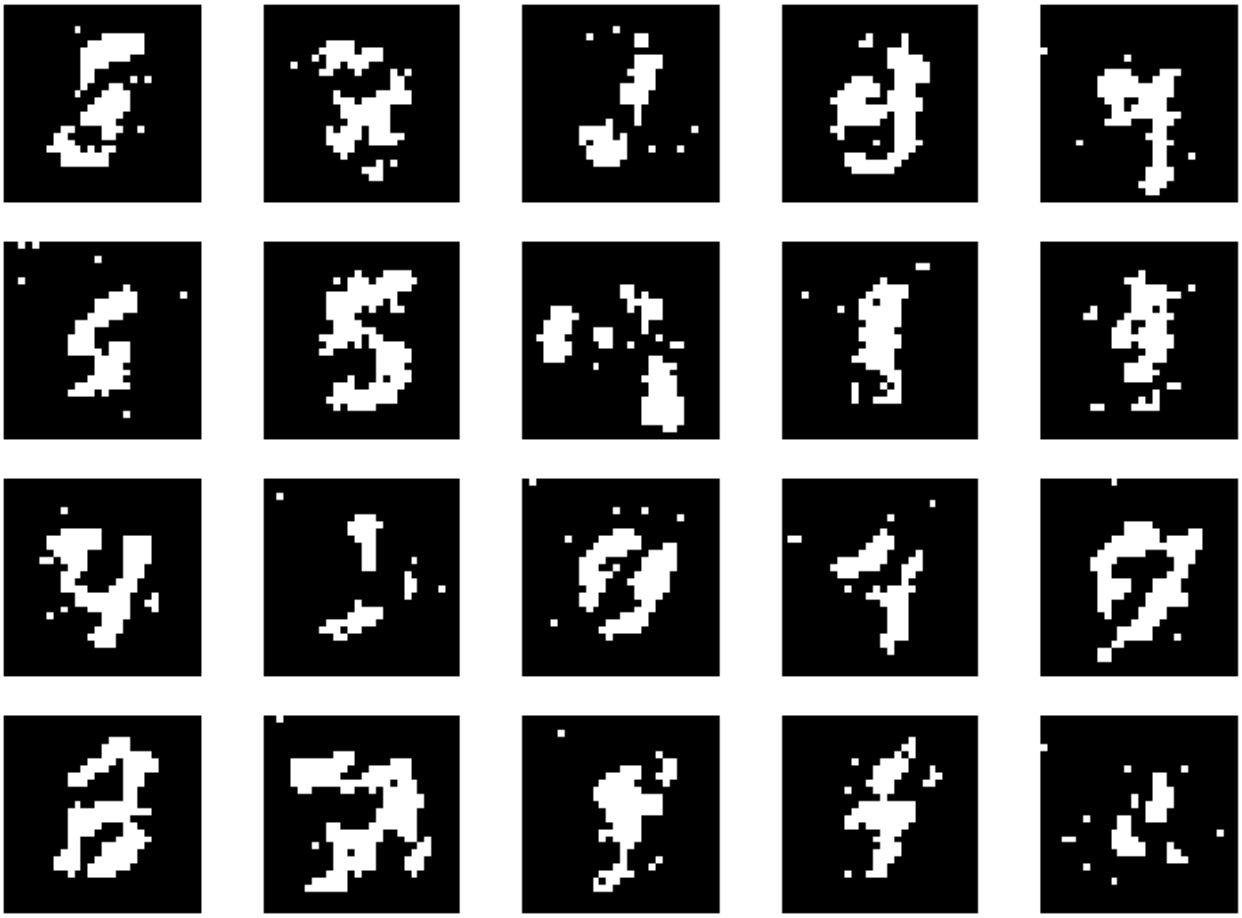


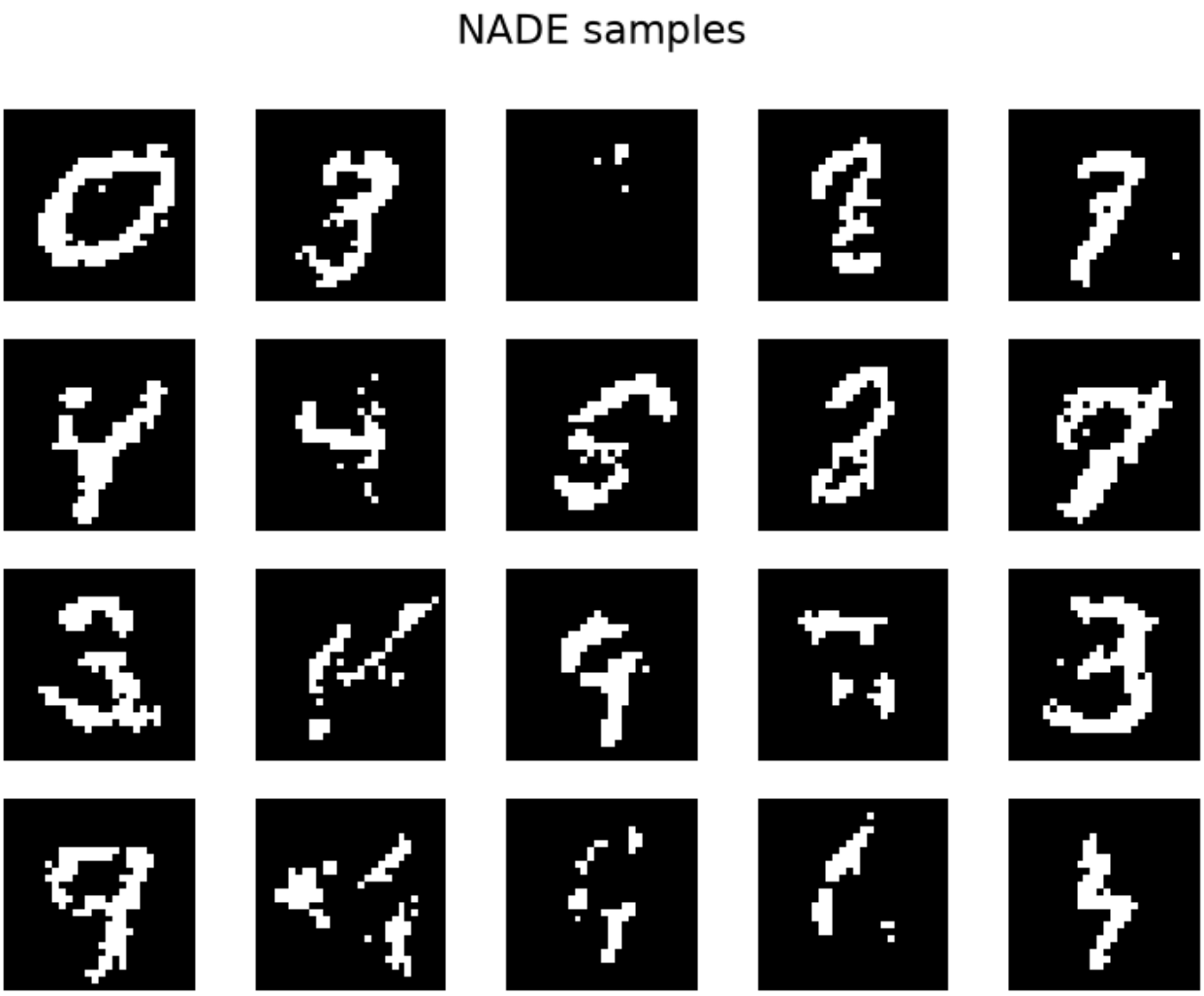
The GAN discriminator and generator losses are shown below.

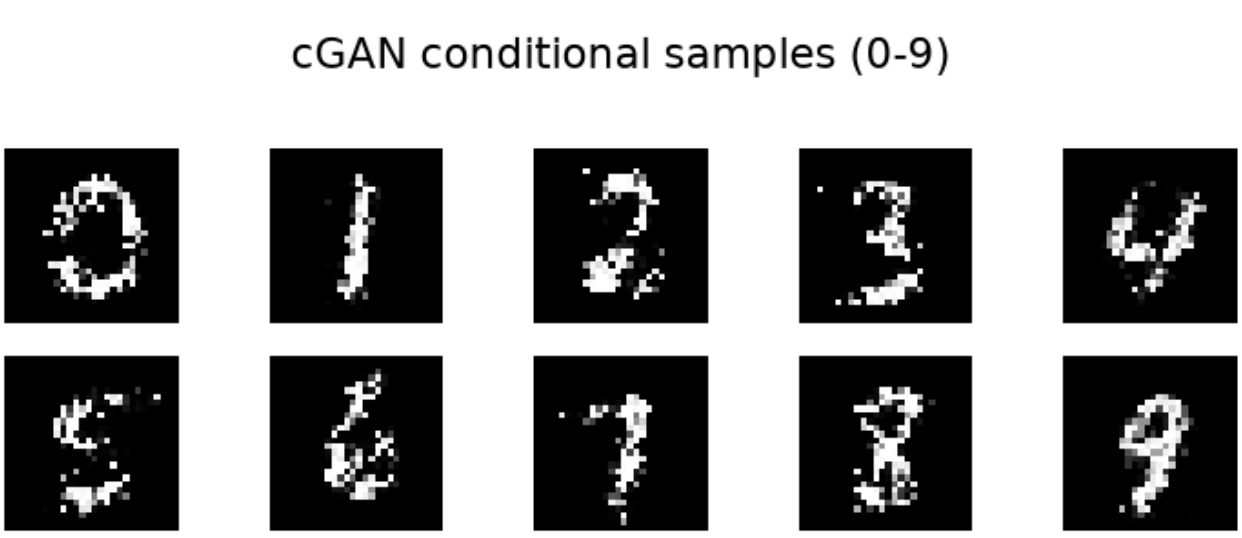


3.3 Generated samples

FVSBN samples

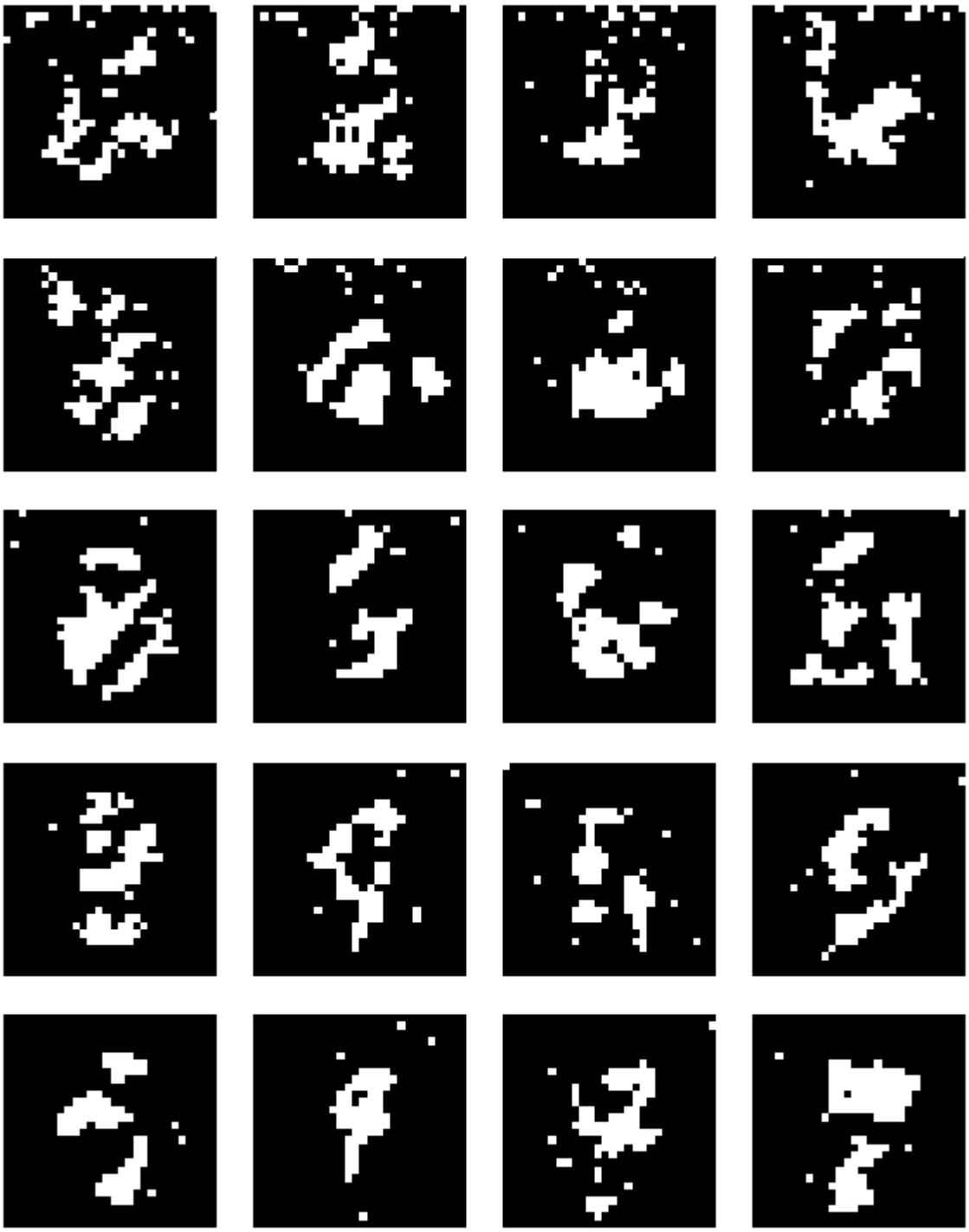


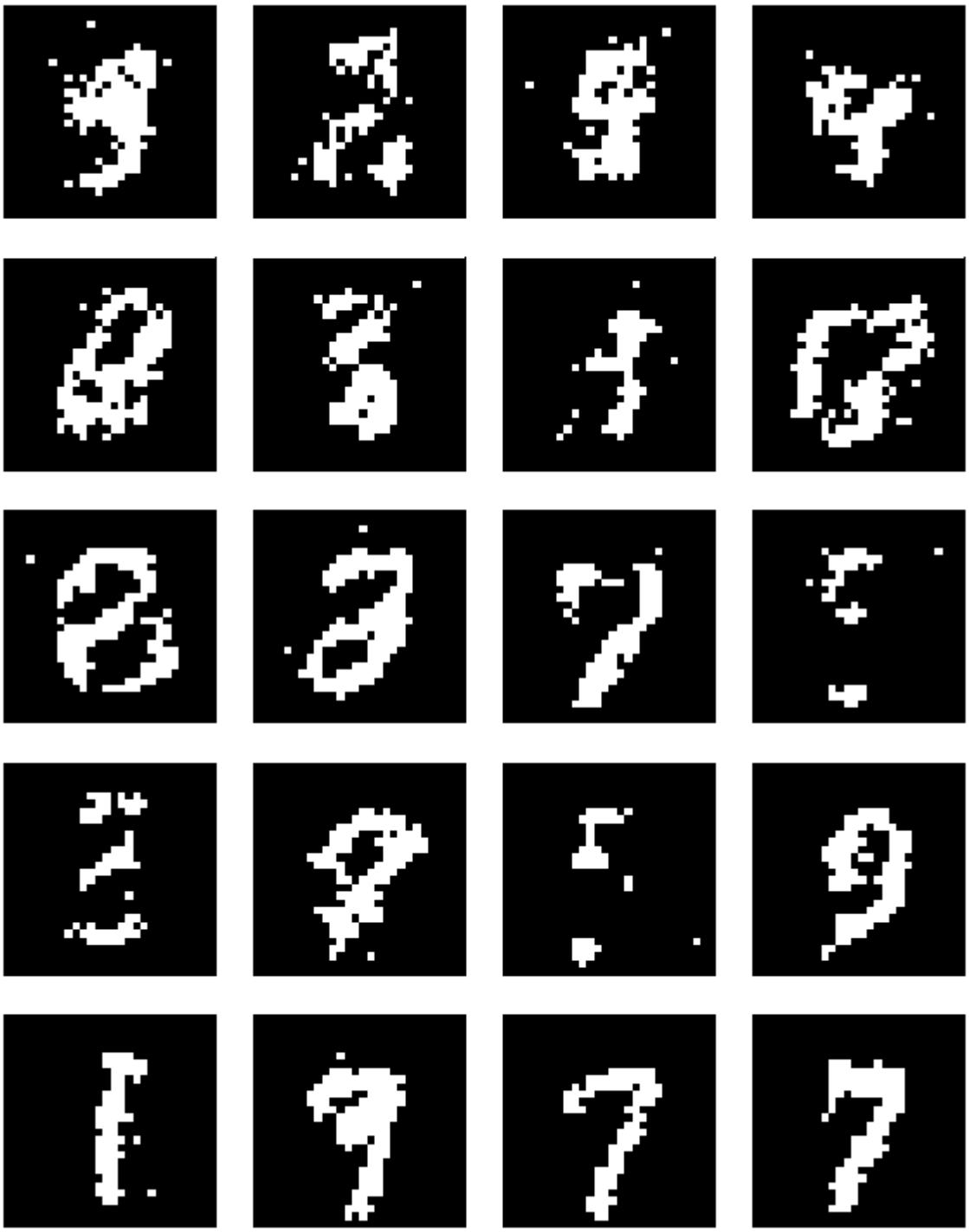


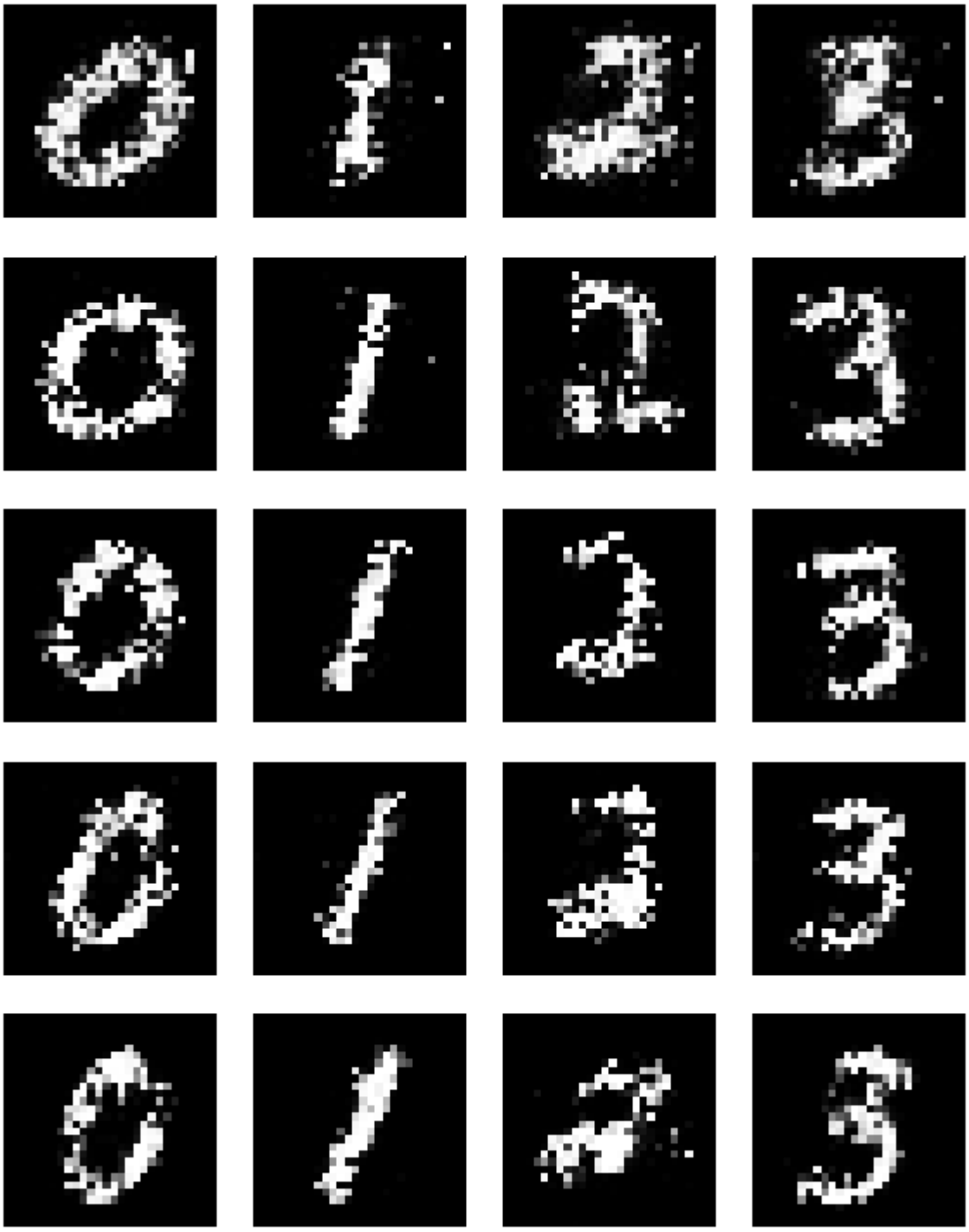


3.4 Training progress

Each row shows samples produced at a progressively later training stage.







3.5 Runtime

All runs use an Apple MPS device. Per-epoch training times are averaged from the training history.

Model	Avg. epoch time	Total training (20 epochs)	Inference (10 samples)
FVSBN	~2.0 s	~40 s	~0.054 s
NADE	~33 s	~660 s	~0.088 s
cGAN	~3.3 s	~66 s	~0.0006 s

3.6 Analysis and discussion

- **FVSBN vs NADE:** After 20 epochs NADE reaches test bpd ~ 0.162 versus FVSBN's ~ 0.182, consistent with NADE's shared hidden representation being a stronger model than FVSBN's independent logistic regressions.
- **GAN stability:** With the vanilla GAN we observed mode collapse - the generator produced only digit 0 after a few epochs, with the discriminator loss collapsing to zero. Adding label conditioning plus the wrong-label penalty (and label smoothing/input noise) kept the discriminator loss near equilibrium and allowed all ten classes to be generated.
- **Conditional generation:** The cGAN can generate a requested digit directly by feeding the corresponding label, whereas FVSBN and NADE are unconditional and only produce random digits.

3.7 GAN training stability: two failure modes observed

We explicitly observed two distinct failure modes during GAN development, and both are worth reporting because they illustrate different ways GAN training can go wrong.

1. **Discriminator saturation (vanilla GAN).** In the first unconditional run the discriminator loss collapsed to ~ 0.0000 by epoch 10 while the generator loss rose to ~ 11.7. With `BCEWithLogitsLoss`, the generator objective -*log*sigma(D(G(z)))* diverges once the discriminator becomes confident that the samples are fake, and the discriminator's own gradients vanish. The two losses therefore moved in opposite directions: `d_loss` went to zero and `g_loss` kept increasing. The diagnosis is that the discriminator overpowered the generator and saturating losses stopped providing a useful training signal.

2. **Mode collapse with healthy-looking losses.** After adding label smoothing (real labels set to 0.9), Gaussian input noise (\s*igma=0.1*) and a smaller discriminator, the curves looked stable - `d_loss` stayed around 0.4-0.9 and `g_loss` around 2-4, and neither diverged. However, the generated images were still effectively identical: a digit classifier assigned all 1000 sampled images to class 0. This is the classic mode collapse and shows that *loss curves alone are not sufficient* to diagnose it; the samples themselves must be inspected.

3. **Resolution.** Converting the model to a label-conditioned GAN and adding a wrong-label penalty (real images paired with an incorrect label are treated as fake) forced the discriminator to actually use the labels. After this change the generator covered all ten classes: for each requested label we generated 100 images and the classifier matched the requested digit for all of them. The corresponding discriminator/generator losses are shown in Figure `gan_loss_curves.png`, and the per-class diversity is visible in `gan_samples.png` and `gan_progress.png`.

4. Code structure and how to run

See README.md for detailed instructions.

5. Conclusion

We implemented three generative models for MNIST: FVSBN and NADE as exact likelihood-based autoregressive models on binarised digits, and a label- conditioned GAN on the original images. All

models generate recognisable digits. The likelihood models provide a quantitative bpd metric, while the cGAN provides conditional control and faster sampling. The main practical lesson is that vanilla GAN training is prone to mode collapse; label conditioning and the wrong-label penalty were effective stabilisers in our experiments.